

INFORMATIKOS FAKULTETAS

T120B165 Saityno taikomųjų programų projektavimas

Projekto „AbsoluteCinema“ ataskaita

Studentas: Gustas Grubliauskas, IFF-3/9

Dėstytojai: Tomas Blažauskas

KAUNAS 2026

Turinys

1. Sprendžiamo uždavinio aprašymas 3
1.1. Sistemos paskirtis 3
1.2. Funkciniai reikalavimai 3
2. Sistemos architektūra 3
2.1. Realizuotas diegimas 4
3. Pasirinktų technologijų aprašymas 5
3.1. Kliento pusė – React.js 5
3.2. Serverio pusė – PHP Laravel 5
3.3. Duomenų bazė – MySQL 6
3.4. Autentifikacija ir autorizacija – JWT 6
3.5. Diegimas – Railway 7
4. Naudotojo sąsajos projektas 8
4.1. Sąsajos eskizai 8
4.2. Dizaino sprendimai 11
4.3. Realizuota naudotojo sąsaja 12
5. API specifikacija 18
5.1. Bendri principai 18
5.2. API metodai 18
5.3. Naudojimo pavyzdžiai 19
5.4. Testavimas 21
6. Išvados 21

1. Sprendžiamo uždavinio aprašymas

1.1. Sistemos paskirtis

Projekto tikslas – sukurti kino ir serialų katalogo platformą „ AbsoluteCinema “, kurioje naudotojai galėtų susipažinti su filmų studijų sukurtais filmais bei serialais, skaityti ir rašyti jų recenzijas, taip padedant kitiems naudotojams priimti sprendimą, kokį filmą žiūrėti.

Veikimo principas – kuriamą platformą sudaro dvi dalys: internetinė aplikacija, kuria naudosis paprasti lankytojai, registruoti nariai bei administratorius, ir aplikacijų programavimo sąsaja (angl. trump. API).

Sistemoje egzistuoja trys hierarchiškai susieti taikomosios srities objektai: Studija → Filmas → Recenzija. Kiekviena studija turi daug jos sukurtų filmų, o kiekvienas filmas gali turėti daug naudotojų parašytų recenzijų.

Registruotas narys, norėdamas naudotis šia platforma, prisiregistruos prie internetinės aplikacijos ir galės naršyti studijų bei filmų katalogą, filtruoti filmus pagal studiją, žanrą ir išleidimo metus, o peržiūrėjęs filmą – parašyti jam recenziją su įvertinimu balais ir tekstiniu komentaru. Administratorius tvirtins naujas studijas ir filmus prieš jiems pasirodant viešame kataloge bei moderuos naudotojų paliktas recenzijas.

1.2. Funkciniai reikalavimai

Neregistruotas sistemos naudotojas (svečias) galės:

1. Peržiūrėti platformos reprezentacinį puslapį su studijų ir filmų sąrašu;
2. Peržiūrėti filmo informaciją bei apie jį paliktas recenzijas;
3. Prisijungti prie internetinės aplikacijos.

Registruotas sistemos naudotojas (narys) galės:

1. Atsijungti nuo internetinės aplikacijos;
2. Prisijungti (užsiregistruoti) prie platformos;
3. Naršyti studijų sąrašą ir peržiūrėti kiekvienai studijai priklausančius filmus;
4. Filtruoti bei puslapiuoti filmų sąrašą pagal studiją, žanrą ir išleidimo metus;
5. Parašyti recenziją pasirinktam filmui:
   5.1. Nurodyti filmo įvertinimą balais (pvz., nuo 1 iki 10);
   5.2. Pridėti tekstinį recenzijos aprašymą;
6. Redaguoti arba ištrinti savo parašytą recenziją;
7. Peržiūrėti kito naudotojo profilį ir jo parašytas recenzijas;
8. Peržiūrėti bendrą filmo įvertinimą, apskaičiuotą pagal visas jam parašytas recenzijas.

Administratorius galės:

1. Pridėti, redaguoti ir šalinti studijas;
2. Pridėti, redaguoti ir šalinti filmus, priskirdamas juos studijoms;
3. Šalinti netinkamo turinio arba nepagrįstas recenzijas;
4. Šalinti naudotojus.

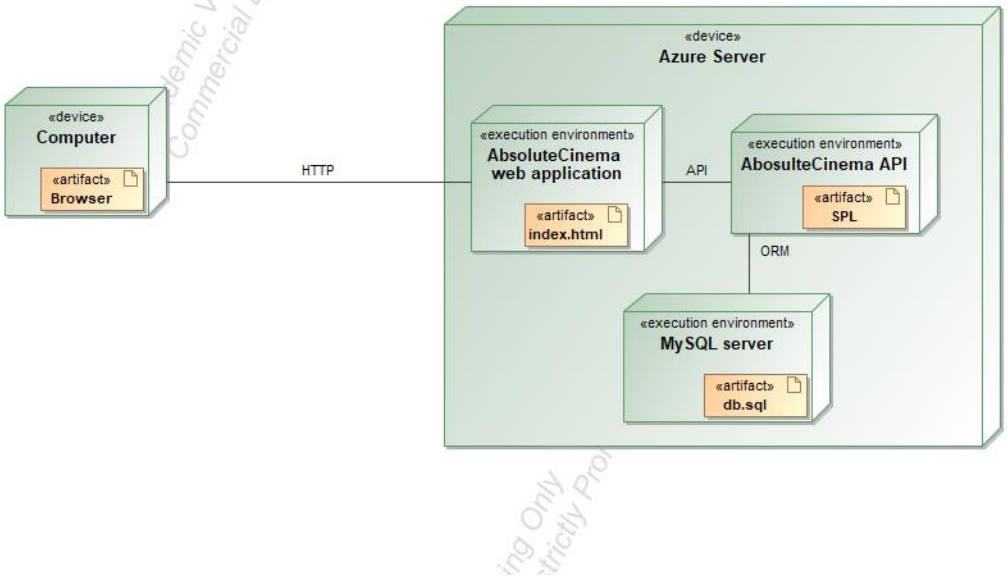
2. Sistemos architektūra

Sistemos sudedamosios dalys:

- Kliento pusė (angl. Front-End) – naudojant React.js;
- Serverio pusė (angl. Back-End) – naudojant PHP Laravel. Duomenų bazė – MySQL.

2.1 pav. pavaizduota kuriamos sistemos diegimo diagrama. Sistemos talpinimui yra naudojamas Railway debesijos platforma. Kiekviena sistemos dalis yra diegiama tame pačiame serveryje. Internetinė aplikacija yra pasiekiama per HTTP protokolą. Šios sistemos veikimui (pvz., duomenų manipuliavimui su duomenų baze) yra

reikalingas AbsoluteCinema API, kuris pasiekiamas per aplikacijų programavimo sąsają. Pats AbsoluteCinema API vykdo duomenų mainus su duomenų baze – tam naudojama ORM sąsaja.



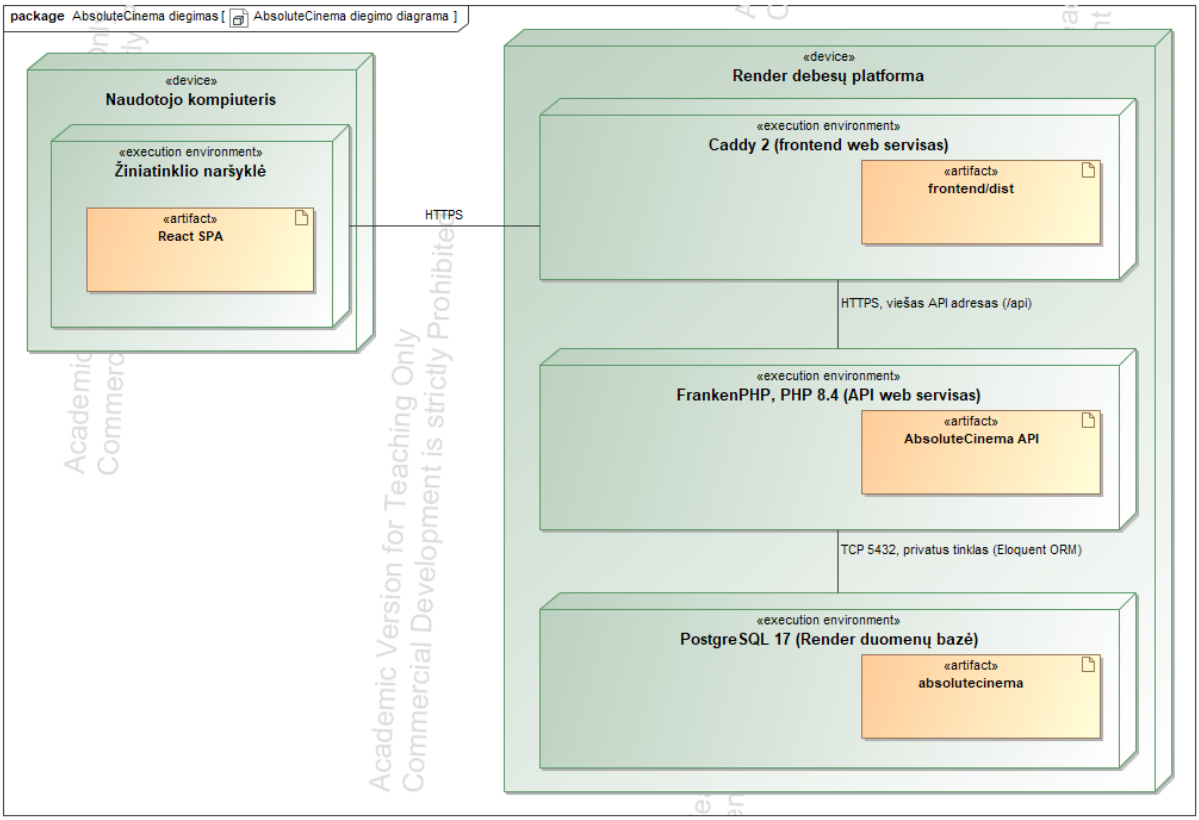
2.1 pav. Sistemos AbsoluteCinema diegimo diagrama

2.1. Realizuotas diegimas

Realizuotos sistemos sudedamosios dalys:

- Kliento pusė – React 19 programa, kurią Vite įrankis sukompiliuoja į statinius failus;
- Serverio pusė – PHP 8.4 ir Laravel 13 REST API;
- Duomenų bazė – PostgreSQL 17 (kodėl ne MySQL – žr. 3.3 skyrių).

Realizuotos sistemos diegimo diagrama pateikta 2.2 pav. Ji sudaryta Magic Systems of Systems Architect įrankiu pagal UML standartą (projekto failas `docs/ataskaita/diegimo-diagrama/AbsoluteCinema_diegimo_diagrama.mdzip`). Sistema diegiama Render debesų platformoje: du nemokami Docker web servisai ir nemokama PostgreSQL duomenų bazė, aprašyti faile `render.yaml`. Naršyklė HTTPS protokolu jungiasi prie frontend serviso, kuriame veikia Caddy serveris. Caddy grąžina React programos failus, o užklausas, kurių adresas prasideda `/api`, persiunčia API servisui jo viešu HTTPS adresu, nes nemokamas Render web servisas negali priimti srauto privačiu tinklu. API servisas veikia FrankenPHP serveryje (PHP 8.4) ir su PostgreSQL duomenų baze dirba per Eloquent ORM privačiu tinklu. Duomenų bazė iš interneto nepasiekiama. Įdiegta sistema pasiekiama adresu https://absolutecinema-6z00.onrender.com, o API – https://absolutecinema-api-1qsh.onrender.com/api/v1.



2.2 pav. Realizuotos sistemos AbsoluteCinema diegimo diagrama

Naršyklė API užklausas siunčia tam pačiam domenui kaip ir svetainė (jas persiunčia Caddy), todėl refresh žetono slapukas naršyklei yra pirmosios šalies slapukas, o CORS konfigūracija nereikalinga. Kiekvienas servisas turi savo Dockerfile, todėl tą pačią sistemą kartu su PostgreSQL konteineriu lokaliai galima paleisti komanda `docker compose up`.

3. Pasirinktų technologijų aprašymas

3.1. Kliento pusė – React.js

Kliento pusei (Front-End) pasirinktas React.js karkasas. React leidžia kurti vartotojo sąsają iš pakartotinai naudojamų komponentų (studijos kortelė, filmo kortelė, recenzijos blokas ir pan.), turi didelę ekosistemą (React Router maršrutizavimui, Axios/Fetch API komunikacijai su serveriu) ir plačią bendruomenę bei dokumentaciją, kas paspartina mokymąsi ir problemų sprendimą.

Realizacijoje naudojama React 19, React Router maršrutizavimui ir Vite kūrimo įrankis. Sąsają sudaro aštuoni puslapiai (pagrindinis, studijų sąrašas, studija, filmas, naudotojo profilis, prisijungimas, registracija ir 404) ir pakartotinai naudojami komponentai: filmo, studijos ir recenzijos kortelės, modalinis langas, puslapiavimas ir formos.

Būsena saugoma trimis lygiais. Komponento duomenys laikomi `useState`, prisijungusio naudotojo duomenys – React Context (`AuthContext`), o filtrai ir puslapio numeris – URL parametruose, todėl filtruotą sąrašą galima išsaugoti nuoroda. Visos užklausos į API siunčiamos per vieną modulį `api/client.js` (Fetch API). Jis prideda access žetoną, o gavęs 401 atsakymą automatiškai atnaujina žetoną ir pakartoja užklausą.

Stiliai parašyti gryna CSS su CSS kintamaisiais, be Bootstrap ar Tailwind. Ikonos – SVG ikonų biblioteka lucide-react. Alternatyvos buvo Vue.js ir Angular.

3.2. Serverio pusė – PHP Laravel

Serverio pusei (Back-End) pasirinktas PHP Laravel karkasas. Laravel turi integruotą Eloquent ORM sąsają, kuri supaprastina darbą su duomenų baze ir hierarchinių ryšių (Studija → Filmas → Recenzija) realizavimą. Laravel

taip pat siūlo paruoštus įrankius maršrutų (routes), validacijos, autentifikacijos (Sanctum/Passport, JWT paketai) ir API resursų (API Resources) formavimui, kas leidžia greitai realizuoti REST principus atitinkantį API.

Realizacijoje naudojamas Laravel 13 (PHP 8.4). Panaudoti šie jo įrankiai: Eloquent ORM modeliams ir jų ryšiams, Form Request klasės duomenų validacijai, Policy klasės autorizacijai, API Resource klasės JSON atsakymams, migracijos ir seeder'iai duomenų bazei. JWT žetonai kuriami ir tikrinami biblioteka firebase/php-jwt.

Visi API maršrutai aprašyti faile `routes/api.php` ir turi priešdėlį `/api/v1`. Prie kiekvieno maršruto nurodyta, ar reikia prisijungimo ir kuri Policy taisyklė tikrinama. Klaidos visam API formuojamos vienoje klasėje (`App\Support\ApiError`) ir grąžinamos RFC 9457 „Problem Details“ formatu. Alternatyvos – Node.js (Express, NestJS), ASP.NET Core ir Django.

3.3. Duomenų bazė – MySQL

Duomenų saugojimui pasirinkta MySQL reliacinė duomenų bazių valdymo sistema. Ji natūraliai atitinka projekto duomenų struktūrą – aiškius, hierarchiškai susijusius objektus (studija, filmas, recenzija) su vienas-su-daug ryšiais, yra gerai palaikoma Laravel Eloquent ORM ir plačiai naudojama pramonėje.

Studijos, filmai ir recenzijos susieti išoriniais raktais su `ON DELETE CASCADE`: ištrynus studiją, ištrinami jos filmai ir jų recenzijos, o ištrynus naudotoją – jo recenzijos. Unikalus indeksas (`movie_id`, `user_id`) neleidžia vienam naudotojui parašyti dviejų recenzijų tam pačiam filmui. Lentelės pateiktos 3.1 lentelėje.

3.1 lentelė. Duomenų bazės lentelės

Lentelė	Paskirtis	Svarbiausi laukai
studios	Filmų studijos	name, description, country, founded_year, headquarters, website, brand_color
movies	Filmai ir serialai (studio_id → studios)	title, type, genre, synopsis, director, release_date, runtime_minutes, seasons, age_rating, poster_url
reviews	Recenzijos (movie_id → movies, user_id → users)	rating, title, body, contains_spoilers, watched_on
users	Naudotojai	name, email, password (bcrypt), role
refresh_tokens	Išduoti refresh žetonai	user_id, token_hash (SHA-256), expires_at, revoked_at
cache	Atšaukti access žetonai ir užklausų limitai	key, value, expiration

Duomenų bazė užpildyta pradiniais duomenimis: 7 realiomis studijomis (Studio Ghibli, Pixar, A24, Warner Bros., HBO, Netflix, Aardman), 40 jų filmų ir serialų, 10 naudotojų ir 120 recenzijų.

Realizacijoje vietoj MySQL naudojama PostgreSQL 17, nes pasirinkta diegimo platforma Render (žr. 3.5 skyrių) MySQL nesiūlo, o PostgreSQL duomenų bazę suteikia nemokamai. Abi DBVS yra reliacinės, todėl lentelių struktūra ir ryšiai nepasikeitė. Kadangi užklausos rašytos per Eloquent, kodą reikėjo pataisyti tik trijose vietose: paieška naudoja `whereLike` (PostgreSQL operatorius LIKE skiria didžiąsias ir mažąsias raides), rikiuojant pagal įvertinimą nurodoma `nulls last`, o ID adrese leidžiamas tik iki 18 skaitmenų (per didelį skaičių PostgreSQL laiko klaida). Kūrimo metu naudota ir SQLite, kad projektą būtų galima paleisti be duomenų bazės serverio. MongoDB atmesta, nes duomenys turi griežtus ryšius tarp lentelių.

3.4. Autentifikacija ir autorizacija – JWT

Naudotojų autentifikacijai ir autorizacijai pasirinkta JWT (JSON Web Token) technologija su access ir refresh žetonų (tokens) strategija: trumpai galiojantis access žetonas naudojamas kasdienėms užklausoms, o ilgiau galiojantis refresh žetonas leidžia atnaujinti access žetoną be pakartotinio prisijungimo. Žetone saugoma naudotojo rolė ir ID, kas leidžia įgyvendinti autorizaciją pagal rolę (svečias, narys, administratorius).

Access žetonas yra JWT, pasirašytas HS256 algoritmu raktu `JWT_SECRET` iš `.env` failo. Jis galioja 15 minučių, siunčiamas antraštėje `Authorization: Bearer` ir turi laukus `sub` (naudotojo ID), `role`, `name`, `typ`, `iat`, `exp`, `jti` ir `iss`. Žetonas sukuriamas klasėje `App\Services\JwtService`, o kiekvienos užklausos metu tikrinamas middleware `AuthenticateJwt`.

Refresh žetonas yra atsitiktinė 64 simbolių eilutė, galiojanti 14 dienų. Naršyklė jį gauna `HttpOnly`, `Secure`, `SameSite=Strict` slapuke, kuris siunčiamas tik į `/api/v1/auth` adresus. Duomenų bazėje saugoma tik

žetono SHA-256 maiša. Kiekvieną kartą atnaujinant senas žetonas atšaukiamas ir išduodamas naujas (rotacija); jei pakartotinai panaudojamas jau pakeistas žetonas, atšaukiamos visos naudotojo sesijos.

Atsijungiant refresh žetonas ištrinamas iš duomenų bazės, o access žetono `jti` įrašomas į atšauktų žetonų sąrašą, kol žetono galiojimas baigsis. Prisijungimo ir registracijos bandymai ribojami iki 10 per minutę.

3.2 lentelė. Rolių teisės

Veiksmas	Svečias	Narys	Administratorius
Peržiūrėti studijas, filmus, recenzijas ir naudotojų profilius	taip	taip	taip
Rašyti recenziją	–	taip	taip
Redaguoti recenziją	–	tik savo	tik savo
Ištrinti recenziją	–	tik savo	bet kurią
Kurti, redaguoti ir šalinti studijas bei filmus	–	–	taip
Šalinti naudotojus	–	–	taip (ne save)

Studijas ir filmus kurti, redaguoti ir šalinti gali tik administratorius, todėl viešame kataloge atsiranda tik jo pridėti įrašai. Rolė saugoma žetone, tačiau kiekvienos užklausos metu sutikrinama su duomenų baze: jei rolė pasikeitė, senas žetonas atmetamas. Sąsaja mygtukus rodo pagal API atsiųstas `_links` nuorodas, todėl rolės kliento pusėje atskirai netikrina.

3.5. Diegimas – Railway

Sistemos diegimui pasirinkta Railway debesijos platforma. Ji leidžia automatiškai diegti tiek Front-End, tiek Back-End dalis tiesiai iš GitHub saugyklos, siūlo valdomą MySQL duomenų bazės paslaugą ir nemokamą planą, pakankamą projekto demonstravimui laboratorinių darbų gynimų metu.

Realizuojant diegimui pasirinkta Render platforma. Railway nemokamai suteikia tik vienkartinį 5 $ kreditą 30 dienų, o vėliau – 1 $ per mėnesį, ko trims servisams nepakanka. Render nemokamai suteikia Docker web servisus ir PostgreSQL duomenų bazę. Visi servisai aprašyti faile `render.yaml` (Render Blueprint), todėl visa sistema sukuriama iš GitHub saugyklos vienu veiksmu. Paleidžiant API konteinerį automatiškai įvykdomos migracijos, o pradiniai duomenys įkeliami tik tada, kai duomenų bazė tuščia.

Nemokamo plano apribojimai: servisai „užmiega“ po 15 minučių be užklausų, o pirmas atidarymas po to trunka apie minutę; nemokama PostgreSQL duomenų bazė galioja 30 dienų nuo sukūrimo. Alternatyvos – Azure App Service ir DigitalOcean.

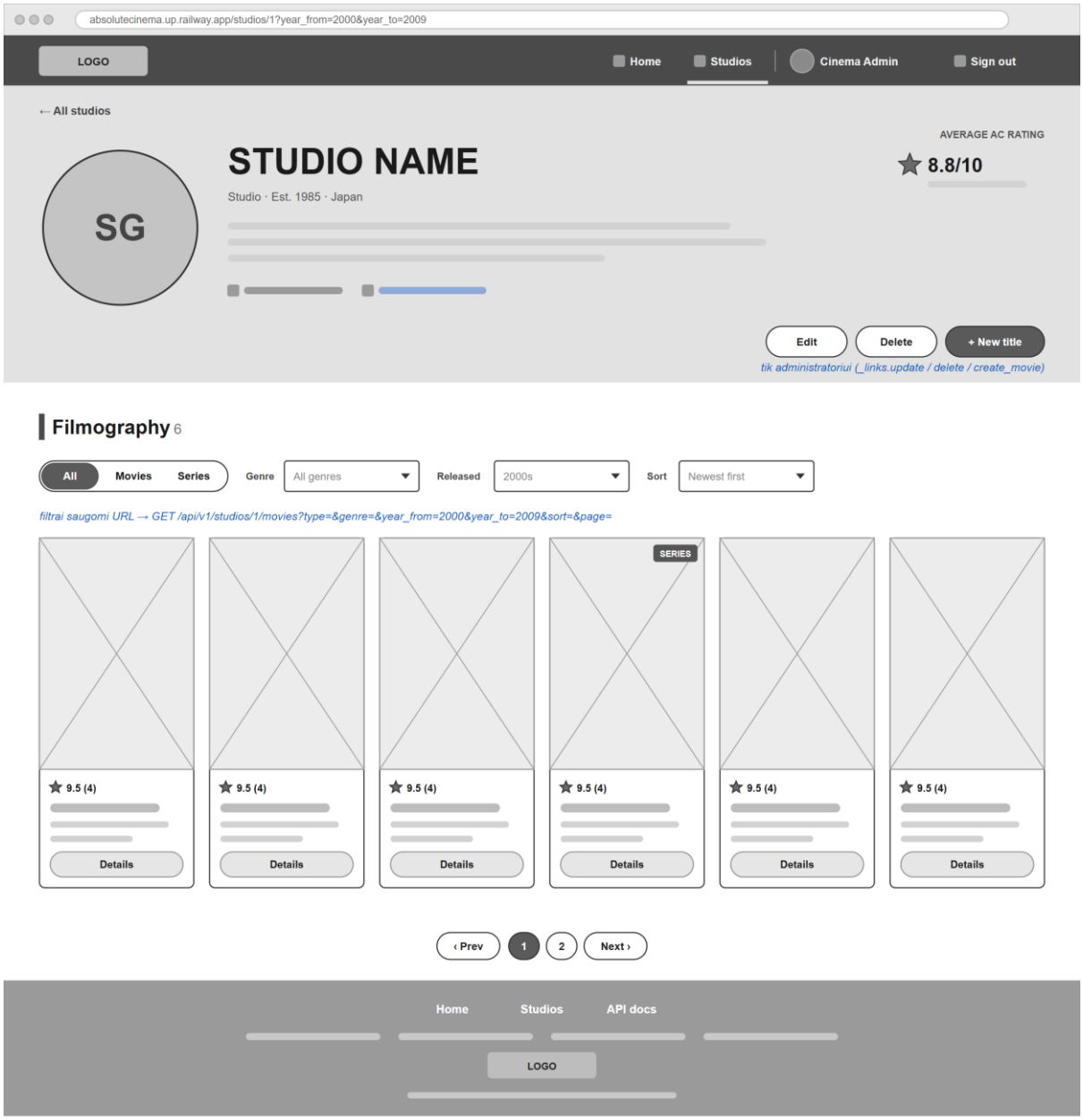
4. Naudotojo sąsajos projektas

4.1. Sąsajos eskizai

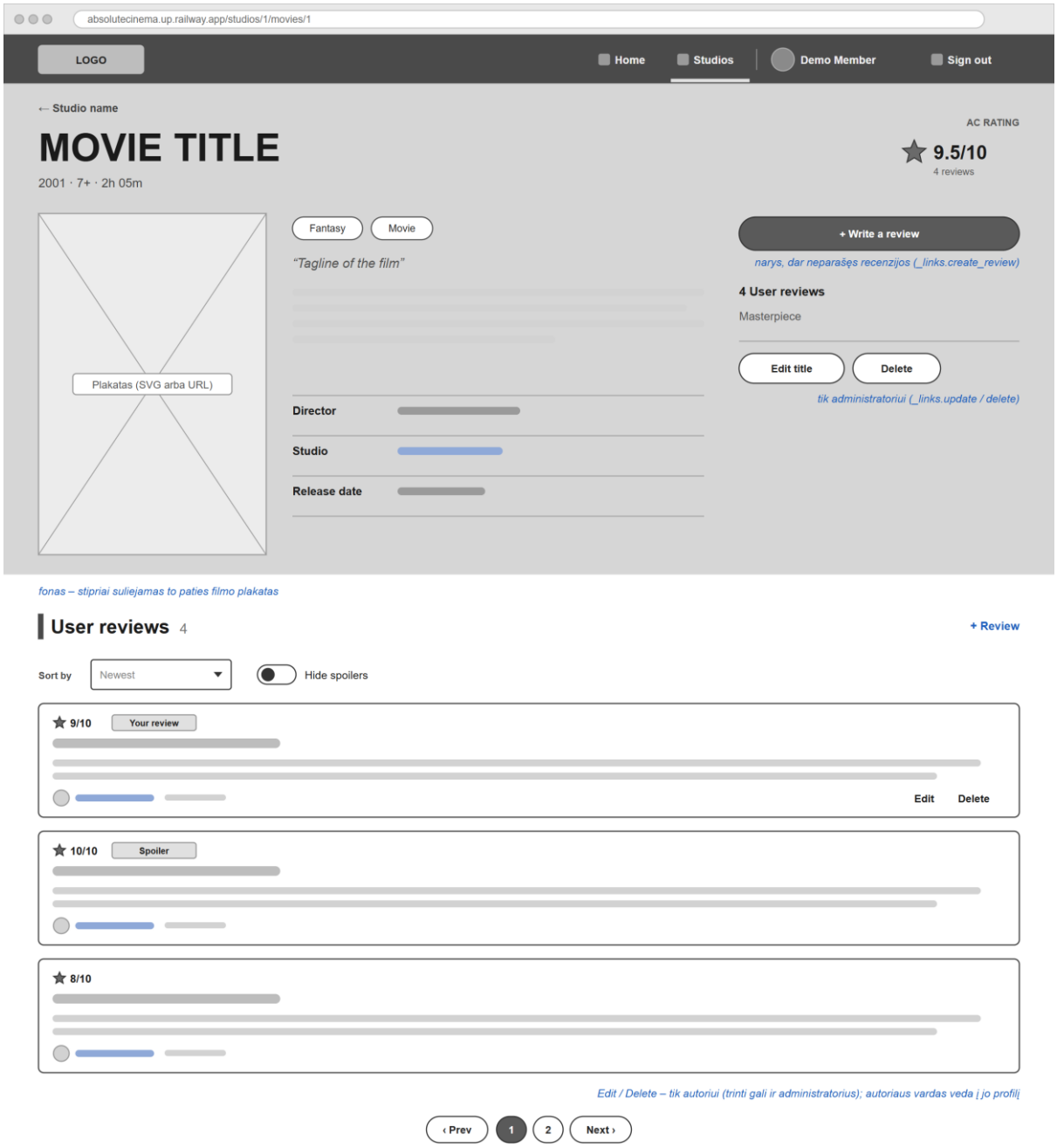
Pagrindinių langų eskizai (wireframe) pateikti 4.1–4.4 pav. Juose parodytas elementų išdėstymas ir veiksmai be spalvų ir šriftų. Mėlynos pastabos nurodo, iš kur gaunami duomenys arba kam matomas elementas.



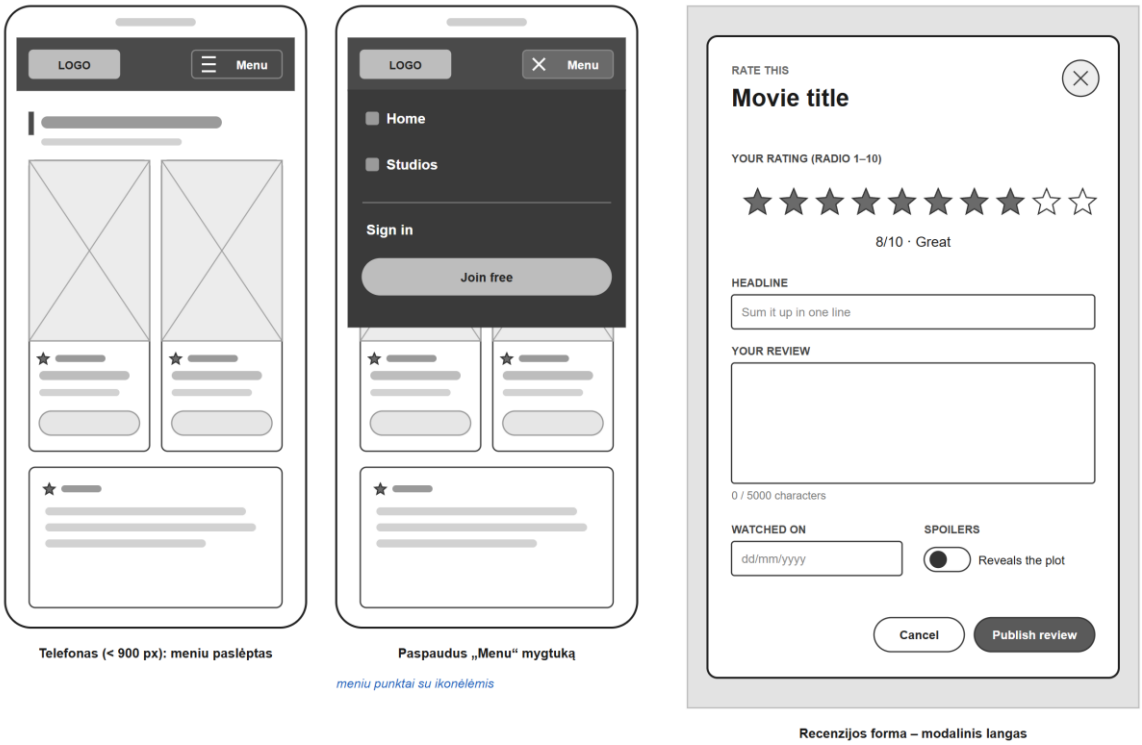
4.1 pav. Pagrindinio puslapio eskizas



4.2 pav. Studijos puslapio su filmų filtrais eskizas



4.3 pav. Filmo puslapio su recenzijomis eskizas



4.4 pav. Mobilaus vaizdo ir recenzijos formos eskizas

4.2. Dizaino sprendimai

Sąsajos stilius artimas filmų katalogams (pvz. IMDb), bet su savu „AbsoluteCinema“ logotipu. Fonas juodas (#000000), kortelės tamsiai pilkos (#1A1A1A), geltona (#F5C518) naudojama logotipui, įvertinimo žvaigždutėms ir pagrindiniams mygtukams, o nuorodos mėlynos (#5799EF). Skyrių antraštės turi geltoną brūkšnį kairėje.

Naudoti Google Fonts šriftai: Roboto visam tekstui ir Big Shoulders Display plakatų pavadinimams. Šriftai įtraukiami faile `index.html`, o jų pavadinimai saugomi CSS kintamuosiuose (`styles/tokens.css`).

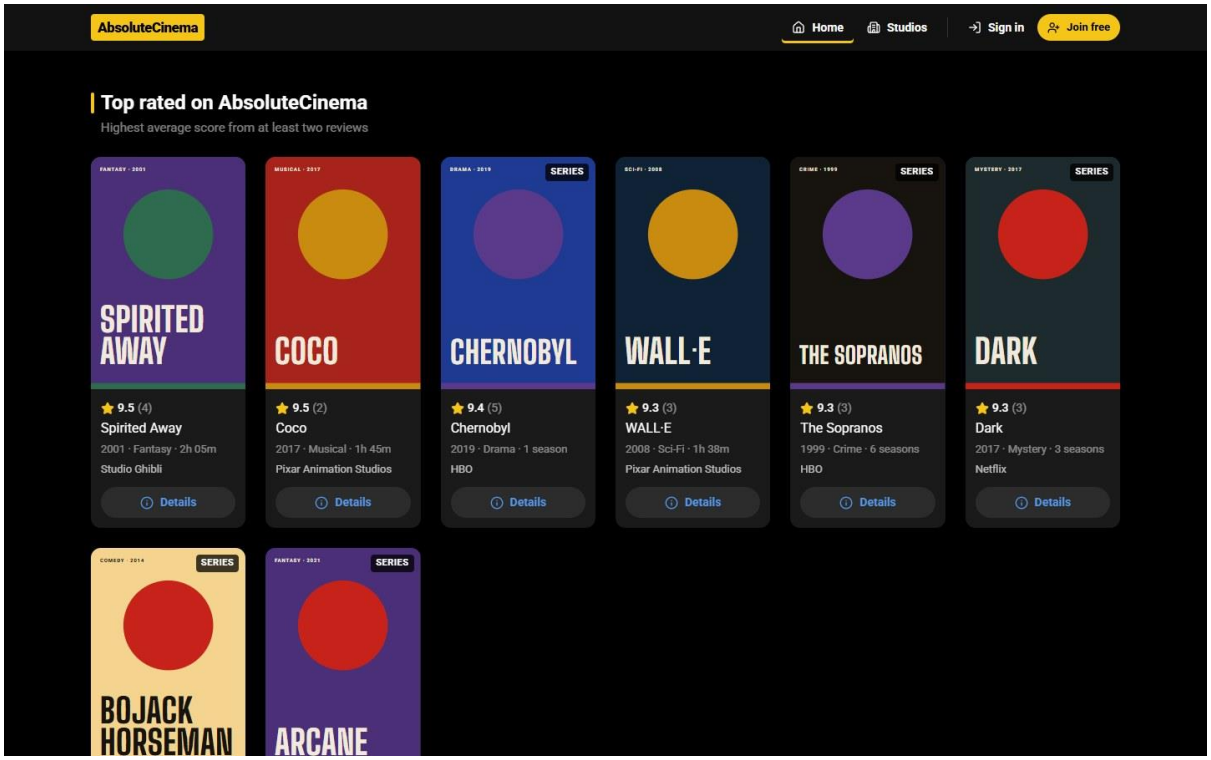
Filmų plakatai, jei nenurodytas paveikslėlio adresas, piešiami SVG formatu: fono spalva priklauso nuo žanro, o apskritimo ir apatinės juostelės spalva – nuo studijos. Sąsaja turi du lūžio taškus: ties 900 px meniu paslepiamas po „Menu“ mygtuku, o ties 600 px elementai išdėstomi telefonui (pvz. du plakatai eilutėje). Reikalavimų realizacija apibendrinta 4.1 lentelėje.

4.1 lentelė. Naudotojo sąsajos reikalavimų realizacija

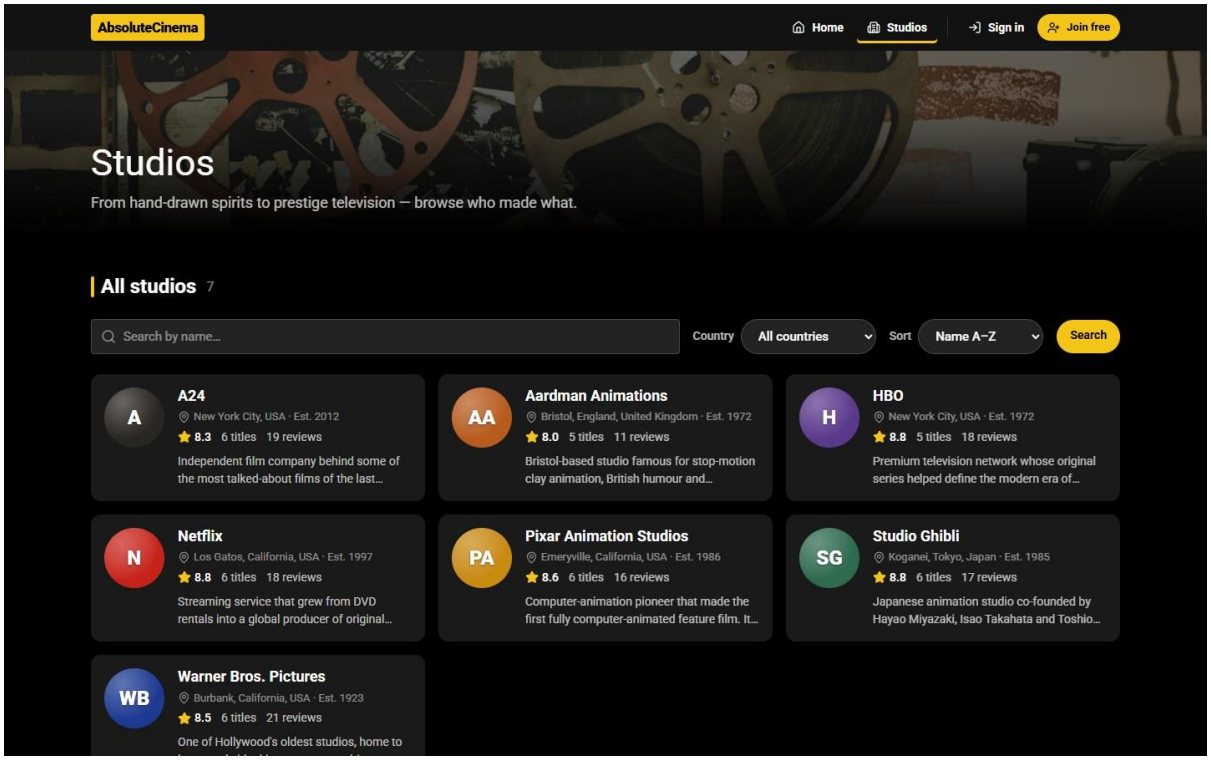
Reikalavimas	Realizacija
CRUD veiksmai per sąsają	Studijos – studijų sąrašo ir studijos puslapiai; filmai – studijos ir filmo puslapiai; recenzijos – filmo puslapis
Registracija, prisijungimas, žetonų atnaujinimas	Prisijungimo ir registracijos puslapiai; žetonas atnaujinamas automatiškai (api/client.js)
Įvesties elementai	text, email, password, search, textarea, number, date, url, color, select, radio, checkbox, datalist
Paveikslėliai	Nuotraukos prisijungimo, registracijos, studijų ir pagrindiniame puslapiuose, SVG plakatai; img { max-width: 100% }
Vektorinės ikonos	lucide-react ikonos meniu punktuose ir mygtukuose
Modaliniai langai	Kūrimo, redagavimo ir trynimo patvirtinimo formos
Animacijos	Puslapio perėjimas, lėtas fono priartėjimas, krovimo žiedas, modalinio lango ir pranešimų atsiradimas, meniu išsiskleidimas, kortelių efektai užvedus pelę (styles/animations.css)
Grįžtamasis ryšys	Pranešimai (toast) ir klaidos prie formos laukų; alert() nenaudojamas
Prisitaikantis išdėstymas	Lūžio taškai 900 px ir 600 px, „Menu“ (hamburger) mygtukas telefone

4.3. Realizuota naudotojo sąsaja

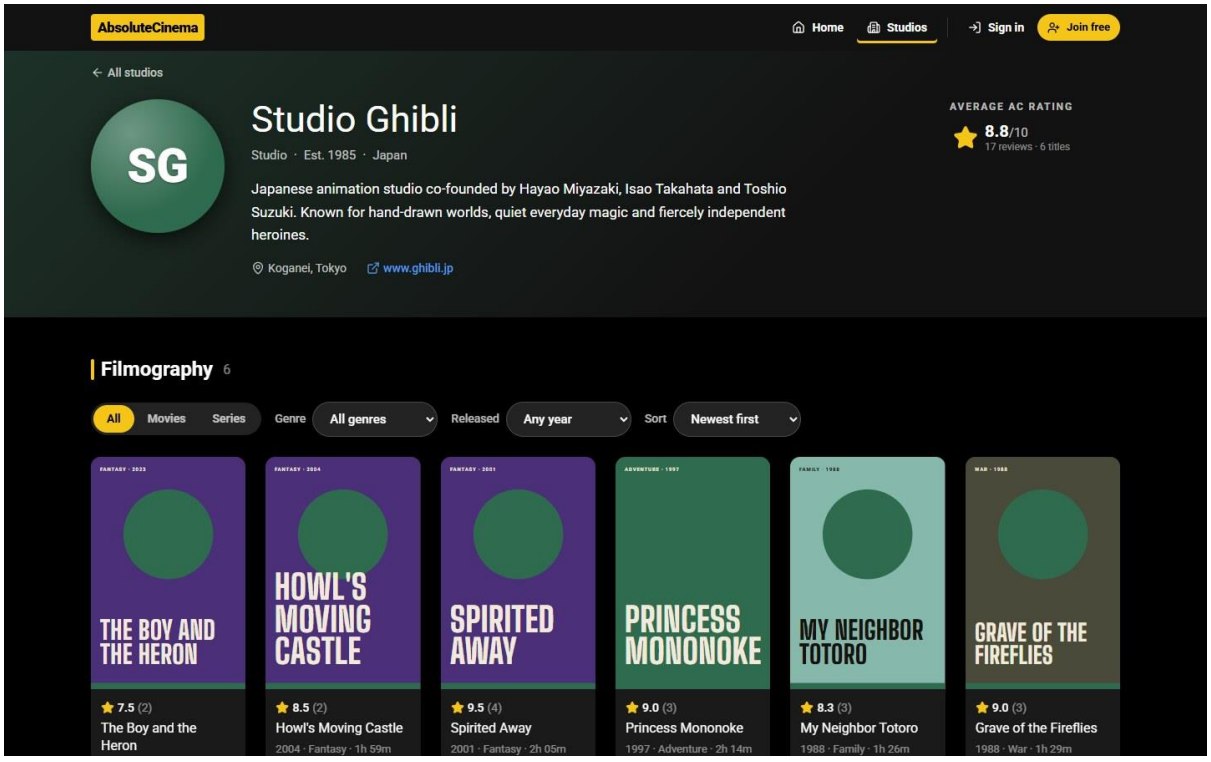
Realizuotos sąsajos ekranvaizdžiai pateikti 4.5–4.14 pav.



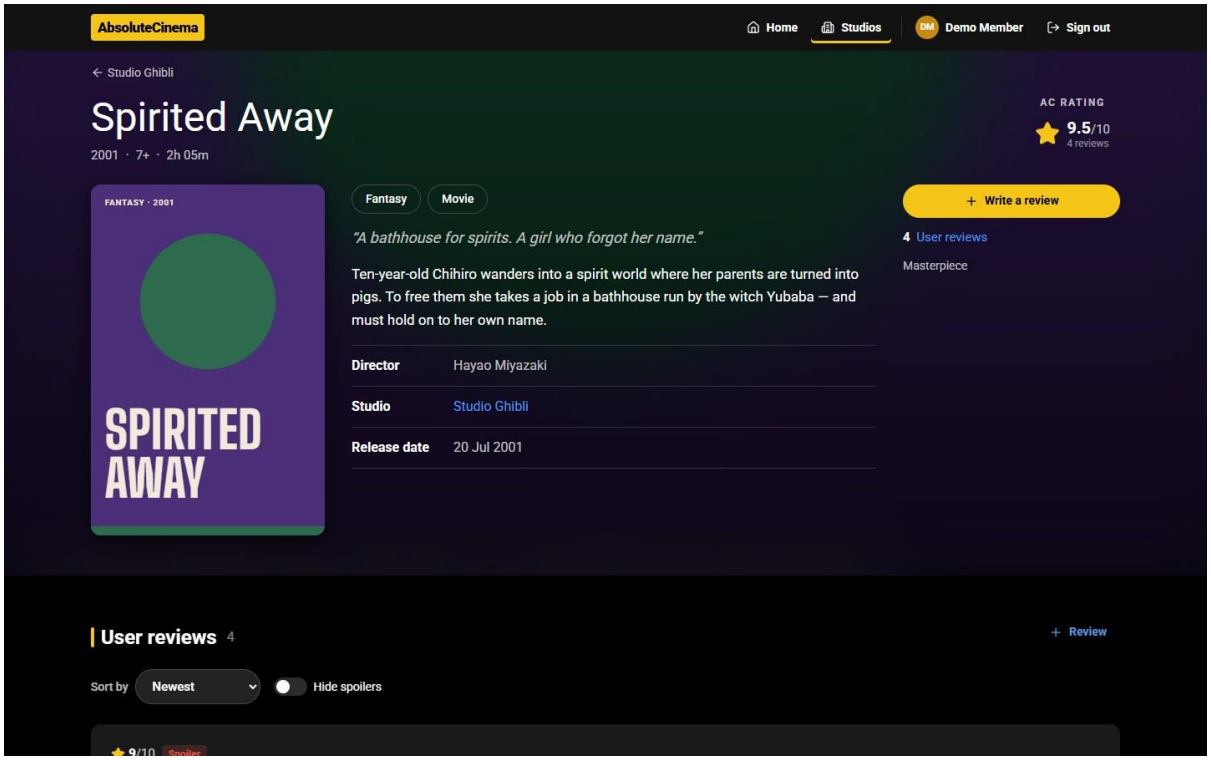
4.5 pav. Pagrindinis puslapis



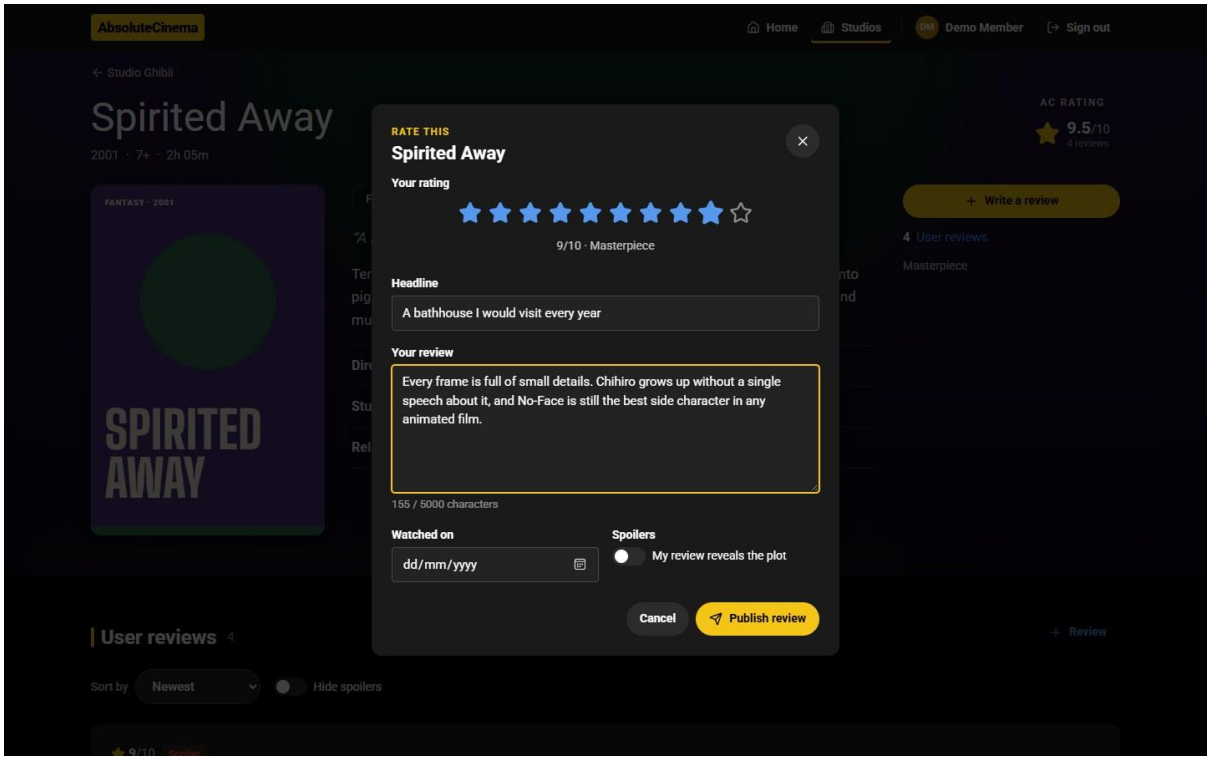
4.6 pav. Studijų sąrašas su paieška ir filtrais



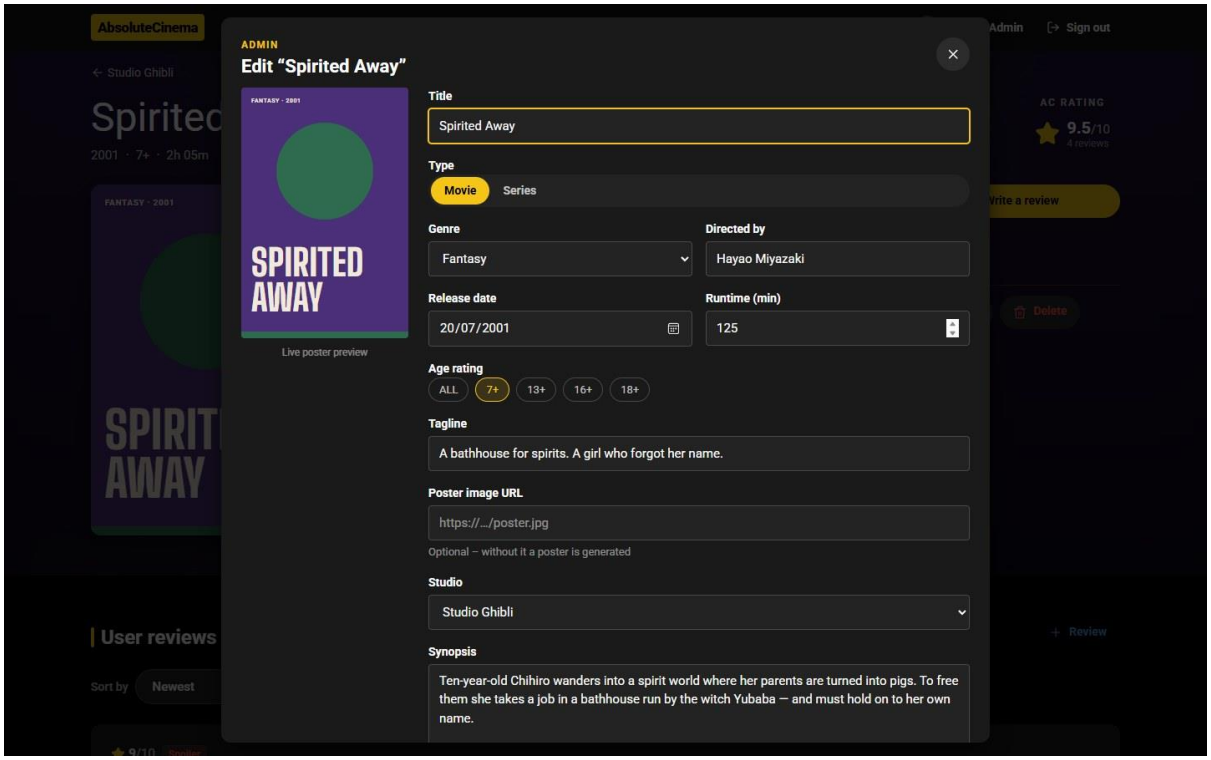
4.7 pav. Studijos puslapis su filmografija ir filtrais



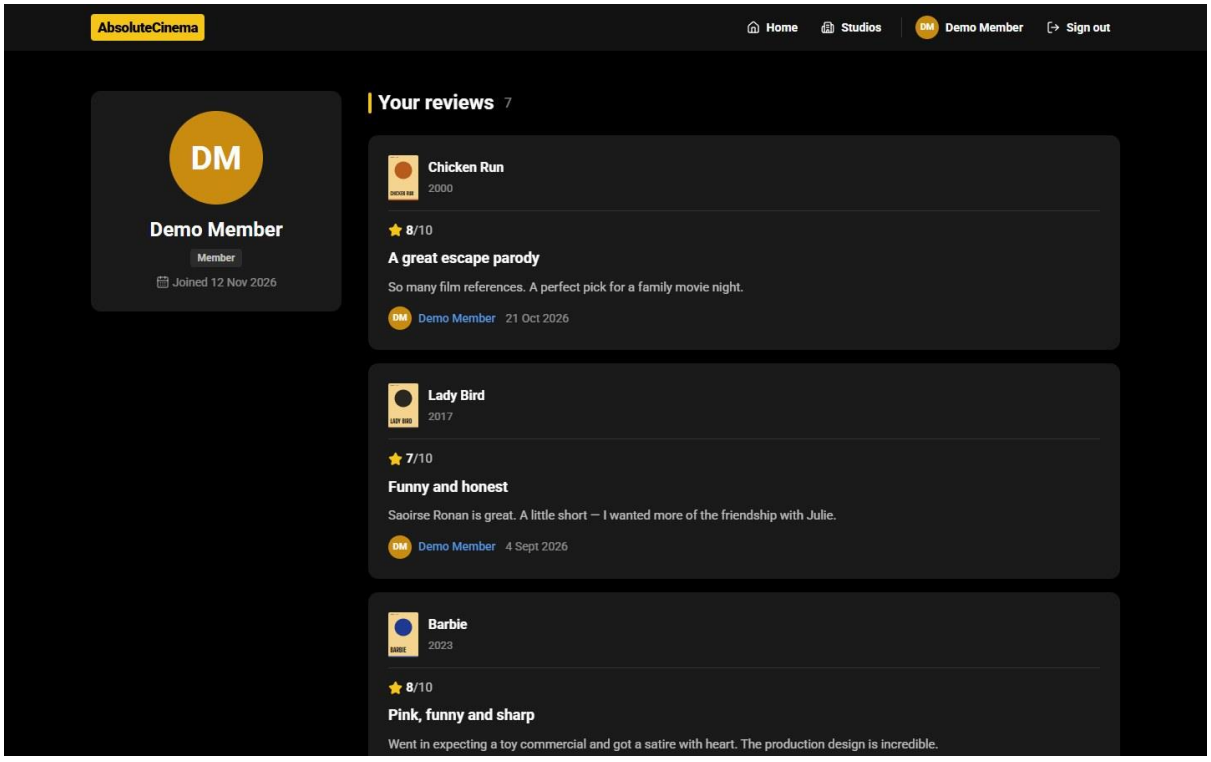
4.8 pav. Filmo puslapis (prisijungęs narys)



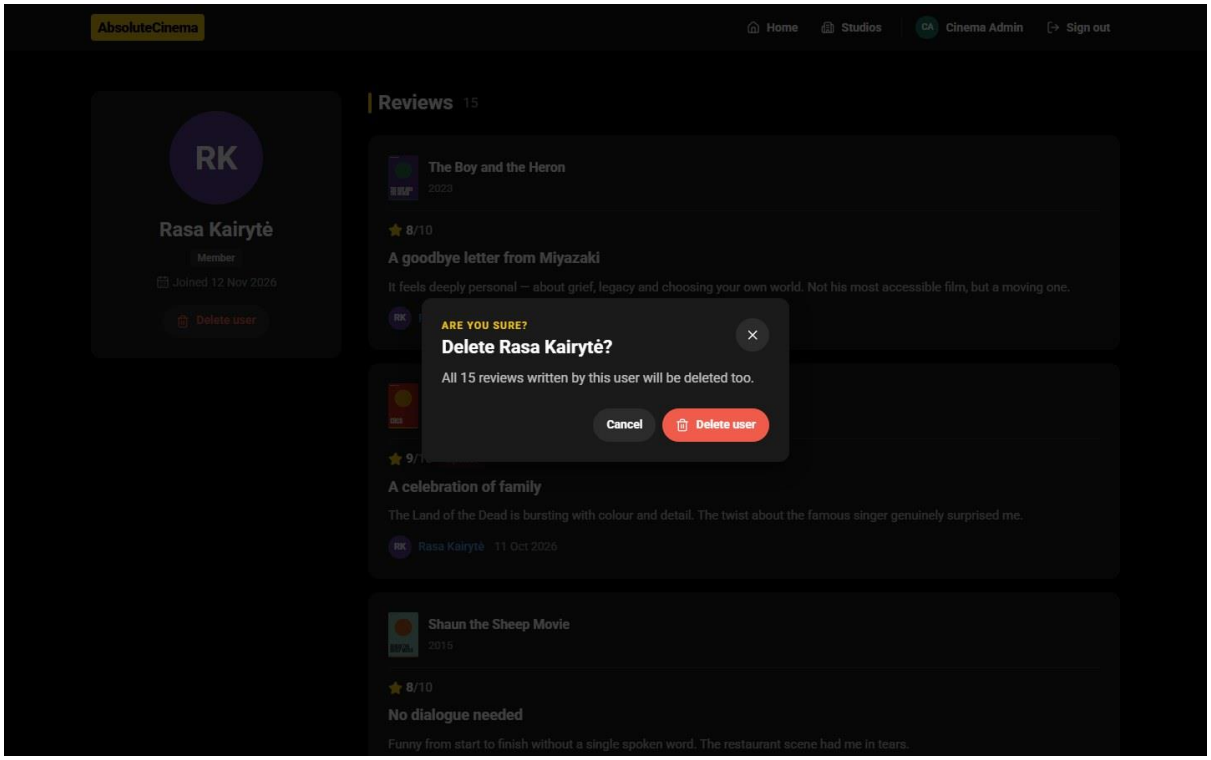
4.9 pav. Recenzijos rašymo forma



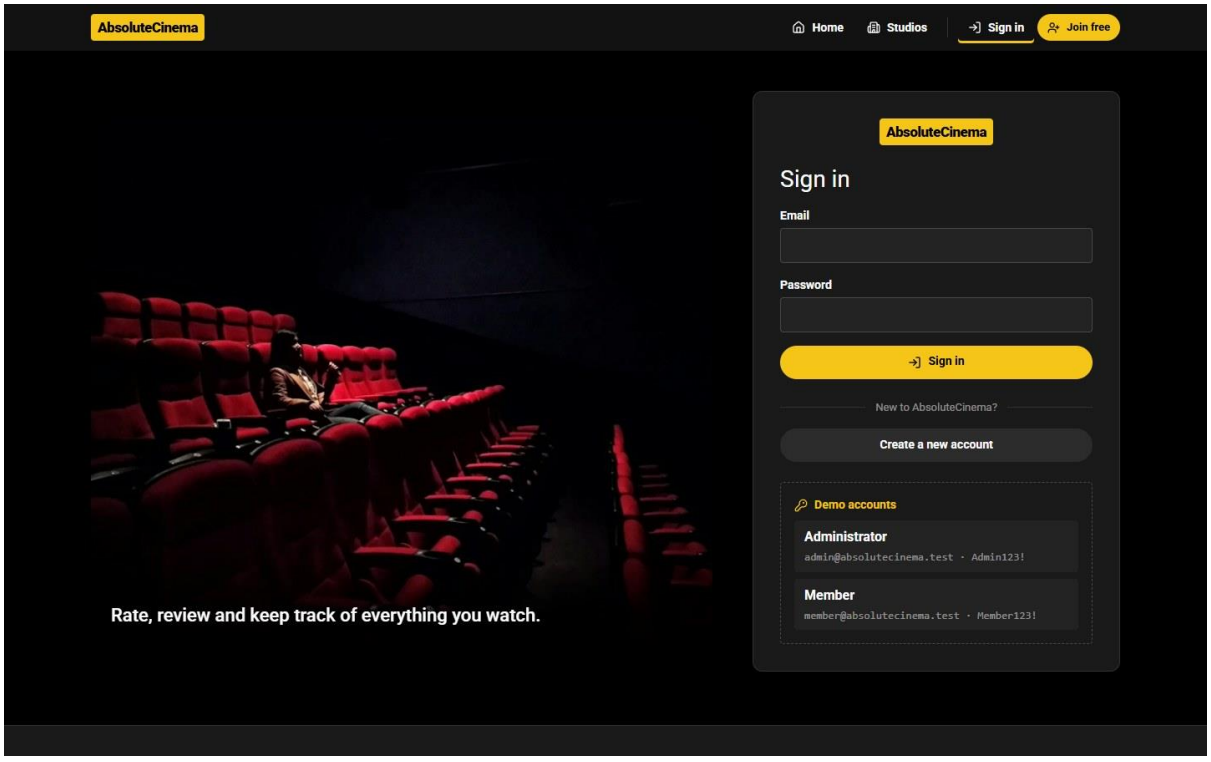
4.10 pav. Filmo redagavimo forma (administratorius)



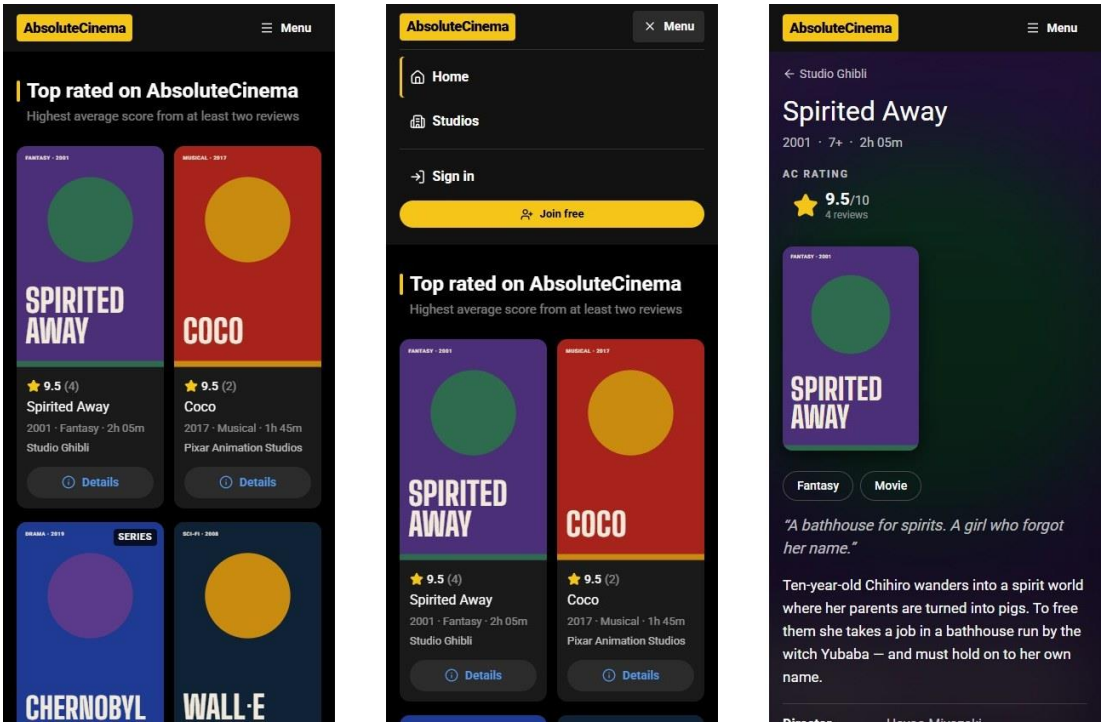
4.11 pav. Naudotojo profilis su jo recenzijomis



4.12 pav. Naudotojo šalinimas (administratorius)



4.13 pav. Prisijungimo puslapis



4.14 pav. Mobilus vaizdas: pagrindinis puslapis, atidarytas meniu ir filmo puslapis

5. API specifikacija

API aprašytas OpenAPI 3.0 specifikacija. Joje kiekvienam metodui pateikti parametrai, užklausų ir atsakymų pavyzdžiai bei galimi klaidų kodai. Specifikacija yra saugykloje (`backend/public/openapi.yaml`), o paleidus API ją galima naršyti Swagger UI sąsajoje adresu `/docs`.

5.1. Bendri principai

- Bazinis adresas `/api/v1`, visi atsakymai pateikiami JSON formatu.
- Prisijungusio naudotojo užklausos turi antraštę `Authorization: Bearer <access žetonas>`.
- Sąrašai puslapiuojami parametrais `page` ir `per_page` (daugiausia 50). Atsakyme pateikiama `meta` (total, last_page) ir `_links` (first, prev, next, last); filtrų parametrai išlieka ir puslapių nuorodose.
- Filmų sąrašas filtruojamas pagal studiją (adrese), žanrą, tipą ir išleidimo metus, pvz. `/studios/1/movies?genre=fantasy&year_from=2000&year_to=2009`.
- Kiekvienas resursas turi `_links`: savo adresą, susijusius resursus ir veiksmus (update, delete, create_review), kuriuos dabartinis naudotojas gali atlikti.
- Klaidos grąžinamos `application/problem+json` formatu su laukais type, title, status, detail, o validacijos klaidų atveju – ir errors.

5.2. API metodai

5.1 lentelėje pateikti visi 23 API metodai. Hierarchinis metodas, apimantis visus tris objektus, yra `GET /studios/{studio}/movies/{movie}/reviews`; jei filmas nepriklauso nurodytai studijai, grąžinamas 404. Statuso kodų reikšmės pateiktos 5.2 lentelėje.

5.1 lentelė. AbsoluteCinema API metodai (adresai be priešdėlio /api/v1)

Metodas	Adresas	Kas gali kviesti	Sėkmė	Klaidos
GET	/dashboard	visi	200	–
GET	/studios	visi	200	400
POST	/studios	administratorius	201	400, 401, 403, 415, 422
GET	/studios/{studio}	visi	200	404
PUT, PATCH	/studios/{studio}	administratorius	200	401, 403, 404, 422
DELETE	/studios/{studio}	administratorius	204	401, 403, 404
GET	/studios/{studio}/movies	visi	200	400, 404
POST	/studios/{studio}/movies	administratorius	201	401, 403, 404, 422
GET	/studios/{studio}/movies/{movie}	visi	200	404
PUT, PATCH	/studios/{studio}/movies/{movie}	administratorius	200	401, 403, 404, 422
DELETE	/studios/{studio}/movies/{movie}	administratorius	204	401, 403, 404
GET	/studios/{studio}/movies/{movie}/reviews	visi	200	400, 404
POST	/studios/{studio}/movies/{movie}/reviews	narys, administratorius	201	401, 404, 409, 422
GET	/studios/{studio}/movies/{movie}/reviews/{review}	visi	200	404
PUT, PATCH	/studios/{studio}/movies/{movie}/reviews/{review}	tik autorius	200	401, 403, 404, 422
DELETE	/studios/{studio}/movies/{movie}/reviews/{review}	autorius, administratorius	204	401, 403, 404
GET	/users/{user}	visi	200	404
DELETE	/users/{user}	administratorius (ne save)	204	401, 403, 404
GET	/users/{user}/reviews	visi	200	400, 404
POST	/auth/register	visi	201	400, 422, 429
POST	/auth/login	visi	200	400, 401, 422, 429
POST	/auth/refresh	turintys refresh slapuką	200	401

Metodas	Adresas	Kas gali kviesti	Sėkmė	Klaidos
POST	/auth/logout	prisijungę	204	401

5.2 lentelė. Naudojami HTTP statuso kodai

Kodas	Kada grąžinamas
200 OK	Sėkmingas skaitymas arba atnaujinimas
201 Created	Sukurtas resursas; antraštėje Location – jo adresas
204 No Content	Resursas ištrintas, atsijungta
400 Bad Request	Sugadintas JSON arba neteisingi užklausos parametrai (pvz. ?page=abc)
401 Unauthorized	Nėra žetono, jis suklastotas, pasibaigęs ar atšauktas; neteisingas slaptažodis
403 Forbidden	Rolė arba savininkas neleidžia atlikti veiksmo
404 Not Found	Resurso nėra arba jis nepriklauso URL nurodytam tėviniam resursui
405 Method Not Allowed	Netinkamas HTTP metodas (atsakyme antraštė Allow)
409 Conflict	Antra recenzija tam pačiam filmui
415 Unsupported Media Type	Užklausos turinys siunčiamas ne JSON formatu
422 Unprocessable Content	JSON teisingas, bet duomenys neatitinka taisyklių; errors – klaidos pagal laukus
429 Too Many Requests	Viršytas prisijungimo bandymų limitas (10 per minutę)

5.3. Naudojimo pavyzdžiai

Prisijungimas. Atsakyme grąžinamas access žetonas, o refresh žetonas nustatomas slapuke:

```
POST /api/v1/auth/login
Content-Type: application/json

{ "email": "member@absolutecinema.test", "password": "Member123!" }

HTTP/1.1 200 OK
Set-Cookie: ac_refresh_token=…; Max-Age=1209600; path=/api/v1/auth; httponly; samesite=strict
{
  "token_type": "Bearer",
  "access_token": "eyJ0eXAiOiJKV1QiLCJhbGciOiJIUzI1NiJ9.eyJpc3MiOiJodHRw…",
  "expires_in": 900,
  "user": { "id": 2, "name": "Demo Member", "role": "member", … }
}
```

Iššifravus žetono vidurinę dalį (payload) matomi laukai:

```
{ "iss": "http://localhost:8000", "sub": "2", "role": "member",
  "name": "Demo Member", "typ": "access", "iat": 1794085583, "exp": 1794086483,
  "jti": "eba17a85-716c-4a7b-98e4-aa788023d48a" }
```

Hierarchinis recenzijų sąrašas su puslapiavimu ir hypermedia nuorodomis (atsakymas sutrumpintas):

```
GET /api/v1/studios/1/movies/1/reviews?per_page=1

HTTP/1.1 200 OK
{
  "data": [{
    "id": 4, "movie_id": 1, "rating": 9,
    "title": "A love story without a single kiss",
    "contains_spoilers": true, "watched_on": "2026-10-17",
    "author": { "id": 9, "name": "Emilija Stankutė" },
    "_links": {
      "self":   { "href": "…/api/v1/studios/1/movies/1/reviews/4", "method": "GET" },
      "movie":  { "href": "…/api/v1/studios/1/movies/1", "method": "GET" },
      "author": { "href": "…/api/v1/users/9", "method": "GET" }
    }
  }],
  "meta": { "current_page": 1, "per_page": 1, "total": 4, "last_page": 4 },
  "_links": {
    "first": { "href": "…/reviews?per_page=1&page=1" },
    "next":  { "href": "…/reviews?per_page=1&page=2" },
    "last":  { "href": "…/reviews?per_page=1&page=4" },
    "movie": { "href": "…/api/v1/studios/1/movies/1", "method": "GET" }
  }
}
```

Recenzijos sukūrimas. Grąžinamas 201 kodas ir antraštė Location (atsakymas sutrumpintas):

```
POST /api/v1/studios/1/movies/3/reviews
Authorization: Bearer eyJ0eXAiOiJKV1Qi…
Content-Type: application/json

{ "rating": 9, "title": "The forest fights back",
  "body": "Huge in scale, yet every character has a reason for what they do.",
  "contains_spoilers": false, "watched_on": "2026-11-01" }

HTTP/1.1 201 Created
Location: http://localhost:8000/api/v1/studios/1/movies/3/reviews/122
{ "data": { "id": 122, "movie_id": 3, "rating": 9,
    "author": { "id": 2, "name": "Demo Member" }, …
    "_links": { "self": {…}, "movie": {…}, "author": {…},
                "update": { "href": "…/reviews/122", "method": "PUT" },
                "delete": { "href": "…/reviews/122", "method": "DELETE" } } } }
```

Nuorodos update ir delete grąžinamos, nes recenziją sukūrė pats naudotojas; kitam nariui jų nėra.

Klaidų atsakymai:

```
GET /api/v1/studios/999
→ 404 { "type": "about:blank", "title": "Not Found", "status": 404,
        "detail": "Studio #999 was not found." }

POST /api/v1/studios/1/movies/3/reviews   { "rating": 11, "title": "x", "body": "short" }
→ 422 { "type": "about:blank", "title": "Unprocessable Content", "status": 422,
        "detail": "The given data was invalid.",
        "errors": { "rating": ["The rating field must be between 1 and 10."],
                    "title":  ["The title field must be at least 2 characters."],
                    "body":   ["The body field must be at least 10 characters."] } }

POST /api/v1/studios   (siunčia narys, ne administratorius)
→ 403 { "type": "about:blank", "title": "Forbidden", "status": 403,
        "detail": "Only administrators can create studios." }

DELETE /api/v1/users/1   (administratorius bando ištrinti save)
→ 403 { "type": "about:blank", "title": "Forbidden", "status": 403,
        "detail": "Administrators cannot delete their own account." }

POST /api/v1/studios/1/movies/2/reviews   (antra recenzija tam pačiam filmui)
→ 409 { "type": "about:blank", "title": "Conflict", "status": 409,
        "detail": "You have already reviewed this title. Edit your existing review instead." }

POST /api/v1/studios/1/movies/3/reviews   { "rating": 8,
→ 400 { "type": "about:blank", "title": "Bad Request", "status": 400,
        "detail": "Malformed JSON: Syntax error." }
```

5.4. Testavimas

Visi metodai patikrinti Postman kolekcija (`postman/AbsoluteCinema.postman_collection.json`). Ją sudaro 53 užklausos ir 114 automatinių patikrinimų, įskaitant klaidų atvejus (400, 401, 403, 404, 405, 409, 415, 422). Kolekcija praeina maždaug per 7 sekundes ir pabaigoje ištrina savo sukurtus testinius duomenis, todėl ją galima leisti pakartotinai. Kolekcija paleista su SQLite, su PostgreSQL Docker konteineryje ir su įdiegtu API Render platformoje – visais atvejais be klaidų. Papildomai parašyta 13 PHPUnit testų (`backend/tests/Feature/ApiTest.php`).

6. Išvados

1. Sukurta kino ir serialų katalogo sistema „AbsoluteCinema“: 23 metodų REST API ir React naudotojo sąsaja, kurioje visiems trims objektams galima atlikti CRUD veiksmus. Hierarchija Studija → Filmas → Recenzija realizuota įdėtiniais URL. Kadangi vaikinis resursas ieškomas tik tėvinio viduje (Laravel scopeBindings), svetimos studijos filmui grąžinamas 404 be papildomų patikrinimų kiekviename kontroleryje.
2. Visos klaidos formuojamos vienoje vietoje, todėl API grąžina vienodo formato atsakymus ir teisingus statuso kodus. Automatiniai testai rado realų trūkumą: tuščias JSON masyvas [], kurį kai kurie klientai siunčia su DELETE užklausa, buvo atmetamas kodu 400. Taisyklė pataisyta – tuščias masyvas laikomas užklausa be turinio.
3. Refresh žetono laikymas HttpOnly slapuke apsaugo jį nuo XSS atakų, bet reikalauja, kad internetinė aplikacija ir API veiktų tame pačiame domene. Dėl to architektūroje atsirado Caddy tarpinis serveris (kūrimo metu – Vite proxy); kitaip naršyklės, blokuojančios trečiųjų šalių slapukus, pavyzdžiui Safari, neleistų atnaujinti sesijos.
4. Refresh žetonų rotacija su pakartotinio panaudojimo aptikimu turi praktinę kainą: kelios naršyklės kortelės arba React StrictMode kūrimo režime gali vienu metu išsiųsti dvi atnaujinimo užklausas, ir antroji atrodytų kaip pavogto žetono naudojimas. Todėl klientas vienu metu vykdo tik vieną atnaujinimą, o serveris dar 30 sekundžių priima ką tik pakeistą žetoną.
5. Rolė įrašyta žetone, bet serveris ją dar sutikrina su duomenų baze. Jei rolė duomenų bazėje pasikeičia, senas žetonas atmetamas, o klientas, gavęs 401, automatiškai gauna naują žetoną su teisinga role. Taip žetone esanti rolė niekada nesuteikia daugiau teisių, nei turi naudotojas.

6. Hypermedia nuorodos su leidžiamais veiksmais (update, delete, create_review) leido sąsajoje nekartoti teisių logikos: mygtukai rodomi tik tada, kai API atsiunčia atitinkamą nuorodą, todėl sąsaja net netikrina naudotojo rolės. Teisės vis tiek tikrinamos serveryje, todėl paslėptas mygtukas nėra vienintelė apsauga.
7. Kūrimo metu naudota SQLite, o diegiant – PostgreSQL. Tik paleidus tą pačią Postman kolekciją ir klaidingų užklausų patikrą su PostgreSQL išryškėjo trys skirtumai, kurių SQLite neparodė: LIKE skiria raidžių dydį, reikšmės NULL rikiuojamos kitaip, o per didelis ID adrese sukelia duomenų bazės klaidą (500). Todėl sistemą verta testuoti su ta pačia DBVS, kuri naudojama diegiant. Projektuojant metų filtrą pastebėta, kad SQLite datas saugo kaip tekstą su laiku, todėl filtras realizuotas funkcija whereYear, o ne datų eilučių palyginimu; kitaip iš rezultatų dingtų gruodžio 31 d. išleisti filmai.
8. Filmų plakatai piešiami SVG formatu iš žanro ir studijos spalvų bei pavadinimo, todėl nereikėjo naudoti autorių teisėmis saugomų paveikslėlių, o plakatas lieka ryškus bet kokio dydžio ekrane.
9. Abiem servisams parašyti Dockerfile ir docker-compose leidžia visą sistemą su PostgreSQL paleisti viena komanda, o render.yaml – tą pačią sistemą sukurti Render platformoje. Pradinių duomenų įkėlimas praleidžiamas, jei duomenys jau yra, todėl pakartotinis diegimas nesukuria dublikatų ir nesustabdo konteinerio.